AI Agents per Scheduling Adattativo in Sanità (MS3)

Tipologie, Strumenti, Deployment e Prototipazione MVP



Obiettivi della Presentazione

Classificazione

Classificare i tipi di agenti AI implementabili per MS3

Strumenti

Elencare strumenti/prodotti utilizzabili (self-hosted vs cloud)

Deployment

Confrontare deployment locale e remoto con criteri pratici

Workflow MVP

Definire un workflow di prototipazione MVP end-to-end

Best Practices

Fornire consigli operativi su privacy, GDPR e scalabilità

1) Tipi di Agenti AI Realizzabili ed Esempi Pratici

In MS3 gli "agenti" coprono funzioni eterogenee: **interazione, orchestrazione, ottimizzazione, simulazione**

Distinzione tra agenti LLM conversazionali, framework di orchestrazione e tool di ottimizzazione/simulazione



TIPO A

Agenti Conversazionali / Interpreti di Feedback

Scopo

- Raccogliere feedback medico in linguaggio naturale
- Tradurre feedback in vincoli/preferenze strutturate per il motore di scheduling

Output atteso

Preferenze e vincoli formalizzati (es. JSON con pesi, penalità, hard constraints)



A) Esempi Pratici (Conversational / Feedback)



LibreChat (self-hosted, ChatGPT-like)

interfaccia feedback medico eseguibile in locale o su server privati



Jan (open-source locale)

esecuzione LLM localmente, minimizzando l'esfiltrazione dei dati



Botpress (low-code, self-hostable)

progettazione chatbot con logica e flussi guidati (preferenze strutturate)

A) Razionale di Utilizzo e Limiti

Perché usarli

- Migliorano UX e riducono attrito nella raccolta feedback
- Supportano interpretazione di linguaggio naturale e standardizzazione input

Limiti

- Non effettuano ottimizzazione scheduling autonomamente
- Necessitano di backend separato (optimizer/solver) per generare schedule

TIPO B

Framework / Agenti di Orchestrazione AI

Scopo

- Costruire agenti che orchestrano moduli multipli: feedback → pianificazione → riflessione → azione
- Gestire tool, memoria, stato conversazionale e workflow multi-step



B) Esempi Pratici (Orchestration Frameworks)



LangChain

composizione di agenti LLM con tools, memoria e workflow programmabili



AutoGPT

agente autonomo che decomprime obiettivi in sotto-task e tenta l'esecuzione end-to-end



CrewAI / AutoGen / Semantic Kernel

multi-agent orchestration con ruoli (planner/researcher/executor)

B) Razionale di Utilizzo e Limiti

Perché usarli

- Abilitano architetture multilivello (multi-agent, tool-calling, memorie)
- Facilitano integrazione tra UI, NLP, optimizer e explanation agent

Limiti

- Richiedono sviluppo, progettazione e test significativi
- Non sono plug-and-play: integrazione necessaria con moduli di ottimizzazione scheduling

TIPO C

Agenti/Framework per Ottimizzazione e Agent-Based Modelling

Scopo

- Ottimizzare e simulare sistemi complessi (scheduling con preferenze, vincoli e dinamiche)
- Non necessariamente basati su LLM: focus su modelli di ottimizzazione e simulazione



C) Esempi Pratici (Optimization / Simulation)



Repast (agent-based simulation toolkit)

modellazione sistemi multi-agente e sperimentazione policy di scheduling



Reinforcement Learning libraries (es. CleanRL)

implementazioni RL per apprendere politiche di assegnamento via reward/penalty

C) Razionale di Utilizzo e Limiti

Perché usarli

- Supporto a simulazioni "what-if" e analisi di policy
- Potenziale elevato per strategie di ottimizzazione avanzate

Limiti

- Integrazione con MS3 non banale (richiede competenze ML e ingegneristiche)
- Necessità di definire reward, vincoli e dataset/simulator realistici



2) Prodotti/Strumenti Utilizzabili

Free / Self-Hosted

Selezione strumenti in base a: privacy, costo, maturità, integrazione, requisiti GDPR

Distinzione tra open-source/local e servizi cloud/managed

A) Open-Source / Local Deployment

A.1 LLM & Conversational AI

LibreChat

UI e backend per conversazione self-hosted

Jan

UI locale per modelli open-source

Botpress

chatbot low-code con flussi guidati e logica custom

A.2 Agent Frameworks

LangChain

toolkit per agenti, workflow e tool integration

AutoGPT

agente autonomo orientato a goal

CrewAI / AutoGen / Semantic Kernel

orchestrazione multi-agent e ruoli modulari

A.3 Simulation / Optimizers



Repast

agent-based simulation



CleanRL

implementazioni RL e baseline per ottimizzazione



A) Pro e Contro Self-Hosted

Pro

- Controllo completo dei dati
- Minore dipendenza da provider esterni
- Adatto a domini con dati sensibili (schedulazione medici)

Contro

- Costi infrastrutturali (HW) e manutenzione
- Richiede competenze ML/DevOps e MLOps

B) Istanze Remote Gratuite / Servizi Cloud

B.1 Modelli Gestiti (ChatGPT / Gemini / Claude)

- Uso per interpretazione feedback e generazione raccomandazioni
- Possibile accesso tramite tier gratuito o crediti di prova

B) Pro e Contro Cloud



Pro

- Setup rapido, time-to-prototype ridotto
- Modelli potenti, aggiornati e scalabili

Contro

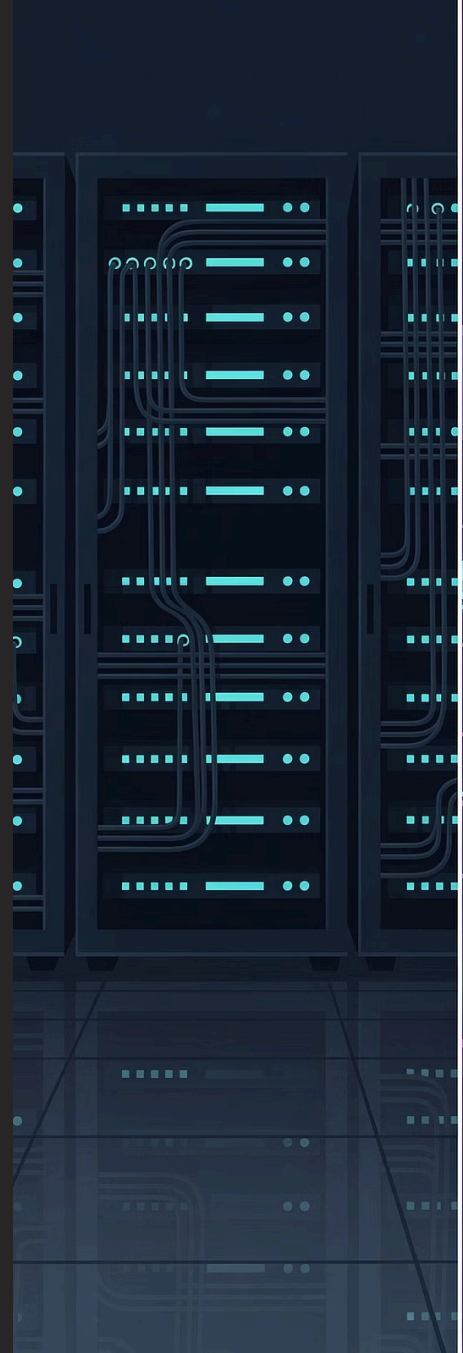
- Anonimizzazione e gestione PII imprescindibili
- Compliance GDPR più onerosa
- Rischio di vendor lock-in e dipendenze esterne

3) Locale vs Cloud

Confronto Pratico per MS3

Criteri decisionali per deployment in funzione di vincoli legali e requisiti tecnici

Valutazione multi-criterio (dati, costi, manutenzione, potenza, compliance)



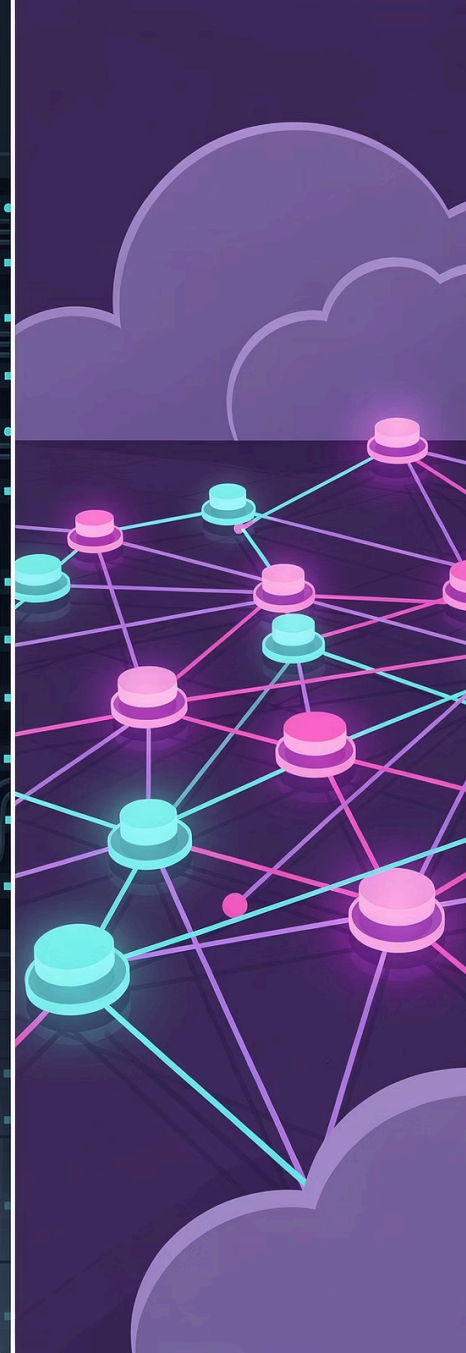


Tabella di Confronto (Criteri Operativi)

Controllo dati locale massimo vs cloud dipendente da policy/contratti	**Setup & manutenzione** locale più costoso vs cloud più semplice	**Potenza modelli** locale vincolato da HW vs cloud più elevata
Privacy/GDPR locale facilitante vs cloud da valutare con attenzione	**Costo totale** locale capex alto vs cloud opex variabile e talvolta gratuito in MVP	



Quando Conviene Locale (Self-Hosted)

Dati sensibili e vincoli legali stringenti

(schedulazione medici)

Necessità di personalizzazioni avanzate e auditabilità

Riduzione rischio compliance

tramite data residency interna

Quando Usare Cloud Gratuito / Managed



Prototipazione rapida e iterazioni veloci



Team con competenze limitate su infrastruttura ML



MVP in fase early-stage con dataset anonimizzati



4) Come Realizzare un Prototipo per MS3

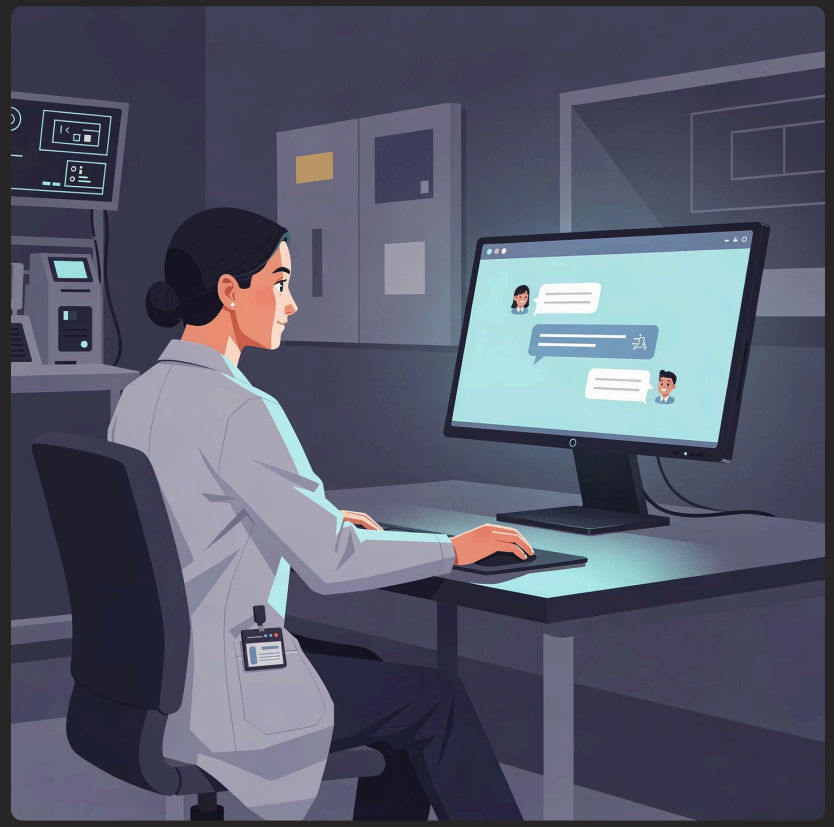
Workflow MVP

Obiettivo: pipeline minima feedback → vincoli → ottimizzazione → spiegazione → loop

Architettura a servizi: UI + interpreter + optimizer + explanation

4.1 Interfaccia Feedback Medico

- Chat UI self-hosted con **LibreChat** o **Botpress**
- Raccolta input libero + opzioni guidate (preferenze strutturate)



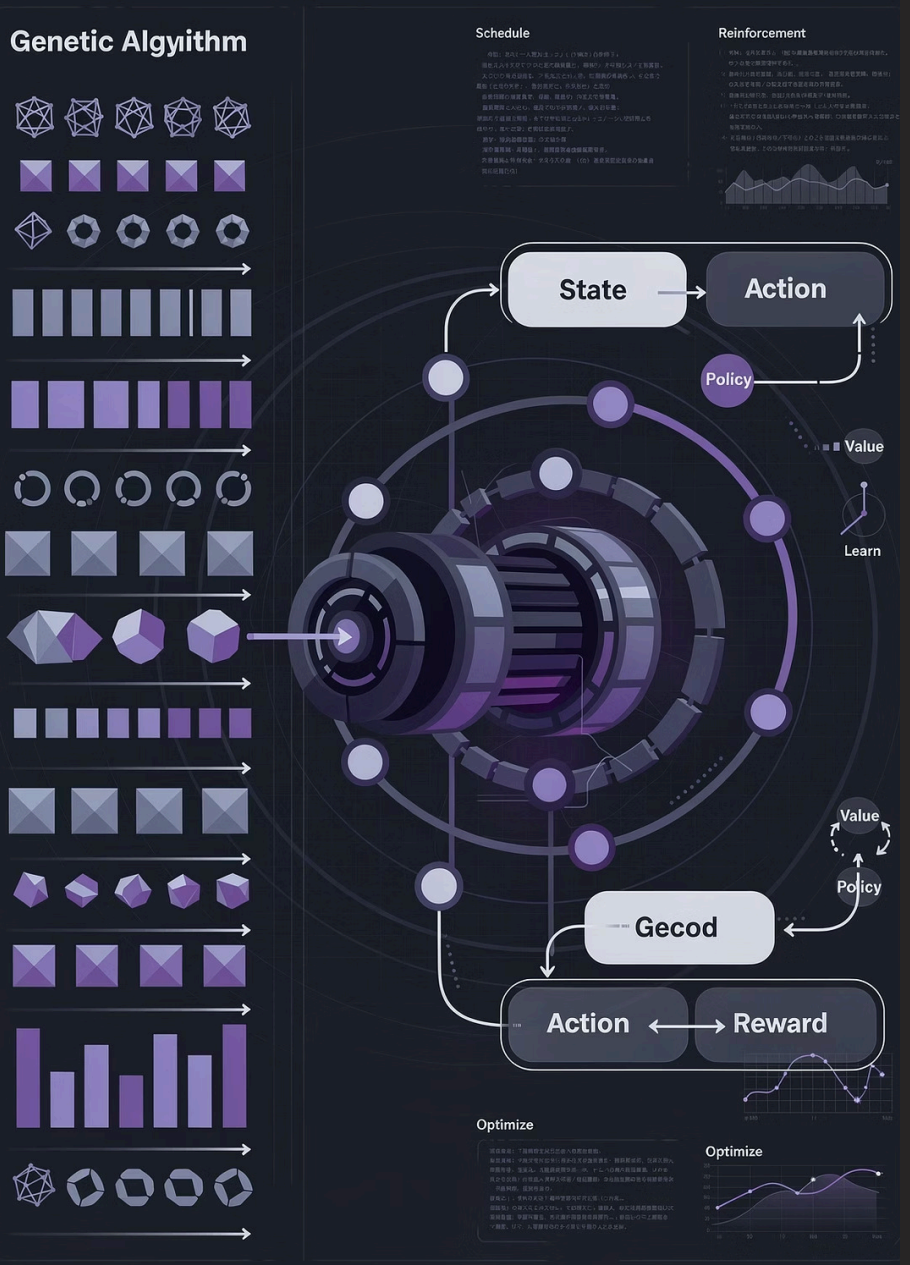
4.2 Traduzione Feedback in Preferenze Strutturate



Interpretazione con LangChain o Jan

Conversione in vincoli/obiettivi

(es. JSON: pesi notti/weekend, hard constraints)



4.3 Motore di Ottimizzazione Scheduling

- Primo step: algoritmo genetico o RL semplice
- Uso di librerie (es. **CleanRL**) per baseline e sperimentazione

4.4 Revisione e Explanation Agent

Agente (es. costruito con LangChain) che:

Propone nuova versione dello schedule

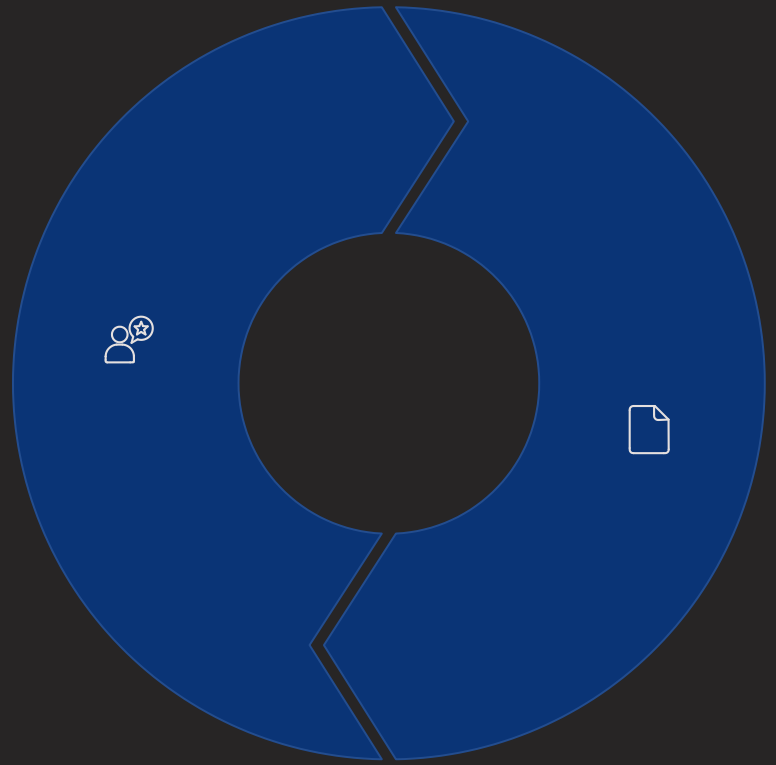
Genera spiegazioni causali

"cosa è cambiato e perché"

4.5 Iterazione con Feedback Medico

Loop continuo

fino a convergenza su qualità/fairness



Tracciamento versioni

di schedule e motivazioni (audit trail)

5) Consigli Operativi

Vincoli reali: privacy, compliance, scalabilità, qualità dati e sostenibilità operativa



Data e Privacy (GDPR)



In cloud

anonimizzazione rigorosa del feedback e assenza di PII



Per dati sensibili

preferenza per locale per ridurre rischio compliance



Controlli

Logging controllato, data retention policy e access control

Scalabilità

MVP

partire con modelli piccoli auto-hosted (Jan, LibreChat)

Evoluzione

potenziamento con modelli cloud per capacità avanzate (ragionamento, robustezza)

Modularità

separare interpreter, optimizer e orchestrator per scaling indipendente

Conclusioni Tecniche

Pipeline modulare

MS3 beneficia di una pipeline agent-based modulare

Architettura integrata

Conversational agent per UX + orchestrazione per workflow + optimizer dedicato per scheduling

Scelta strategica

Scelta locale vs cloud guidata da GDPR, sensibilità dati e capacità infrastrutturali

Approccio iterativo

MVP iterativo con JSON come interfaccia comune tra moduli